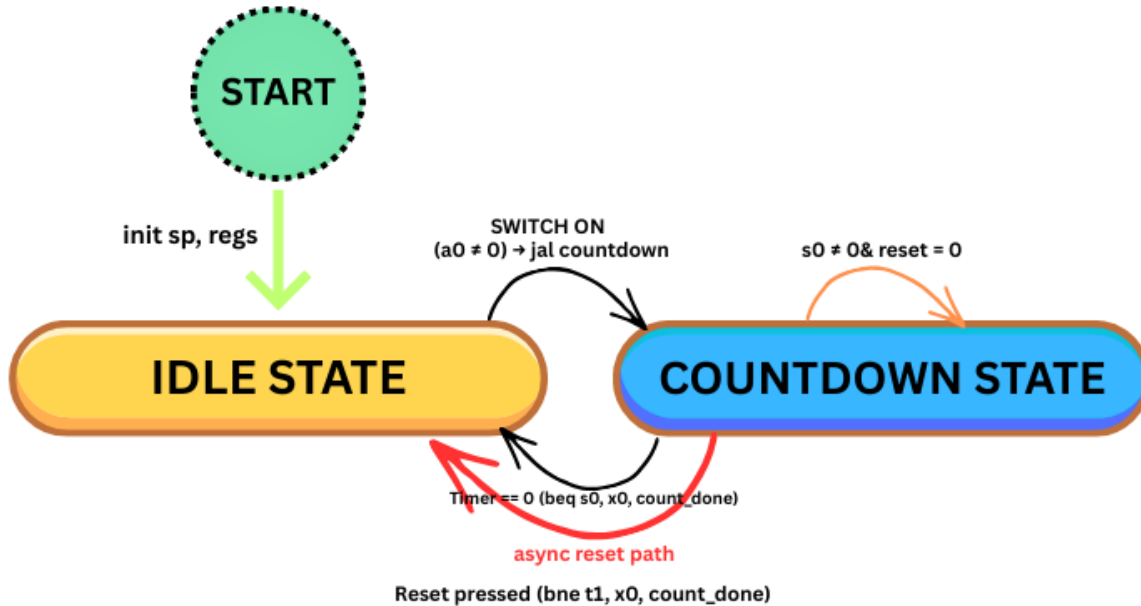


Lab 10: State-Based Control Flow Using RISC-V Assembly

Task 1: FSM Design in Assembly



Description:

The FSM consists of two states: IDLE and COUNTDOWN. On startup, the system initializes the stack pointer and registers, then enters the IDLE state where it continuously polls the switch input (lw a0, SWITCH_ADDR). As long as the switch reads zero, the FSM remains in IDLE.

When a non-zero switch value is detected ($a0 \neq 0$), the captured value is written to the LEDs and the FSM transitions to the COUNTDOWN state via a subroutine call (jal countdown). In COUNTDOWN, the counter register s0 decrements on each iteration and the current value is displayed on the LEDs. The FSM loops back within COUNTDOWN as long as $s0 \neq 0$ and the reset button is not pressed.

Two conditions return the FSM to IDLE: the counter reaching zero (beq s0, x0, count_done) or the reset button being pressed at any point during the countdown (bne t1, x0, count_done). The red reset path is asynchronous meaning it is checked at the top of every loop iteration, ensuring an immediate response.

Task 2: Assembly Implementation of the FSM

```

1  # --- Initialization ---
2  addi x28, x0, 5      # x28 = 5 (Initial value/constant)
3  addi x2, x0, 511     # x2 = 511 (Initialize Stack Pointer or memory limit)
4  addi x5, x0, 768     # x5 = 768 (Base address for INPUT, e.g., Switches)
5  addi x6, x0, 512     # x6 = 512 (Base address for OUTPUT, e.g., LEDs/Display)
6  sw x28, 0(x5)        # Store 5 into input address (simulating initial input)
7
8  # --- Main Input Loop ---
9  INPUT_WAIT:
10     sw x0, 0(x6)      # Clear the output display (write 0 to x6)
11
12  POLL_SWITCH:
13     lw x11, 0(x5)     # Read value from input device into x11
14     beq x11, x0, POLL_SWITCH # If input is 0, keep polling (wait for input)
15
16     add x10, x11, x0 # Copy input value to x10 (argument for countdown)
17     jal x1, COUNTDOWN # Jump to COUNTDOWN, save return address in x1
18     beq x0, x0, INPUT_WAIT # After countdown finishes, loop back to wait for next input
19
20  # --- Countdown Subroutine ---
21  COUNTDOWN:
22     addi x2, x2, -8    # Decrement stack pointer (allocate 8 bytes)
23     sw x1, 4(x2)       # Save return address (x1) to stack
24     sw x12, 0(x2)      # Save x12 to stack (to preserve its value)
25     add x12, x10, x0   # Load the countdown start value into x12
26
27  COUNT_LOOP:
28     sw x12, 0(x6)      # Write current count to output display
29     beq x12, x0, COUNT_DONE # If count reaches 0, exit loop
30     addi x12, x12, -1  # Decrement the counter
31
32     # --- Nested Delay Loop ---
33     addi x13, x0, 3    # Set delay duration (very short here, 3 cycles)
34  DELAY:
35     addi x13, x13, -1  # Decrement delay counter
36     bne x13, x0, DELAY # Loop until delay reaches 0
37
38     beq x0, x0, COUNT_LOOP # Repeat the count loop
39
40  COUNT_DONE:
41     sw x0, 0(x6)      # Ensure display is cleared at the end
42     lw x12, 0(x2)      # Restore x12 from stack
43     lw x1, 4(x2)       # Restore return address (x1) from stack
44     addi x2, x2, 8     # Deallocate stack space
45     jalr x0, 0(x1)     # Return to caller (INPUT_WAIT)
46
47  # --- Final Program End ---
48  end:
49     j end             # Infinite loop to stop execution

```

(Comments)

0x00000300 (Input): This is the "Switch" address; the program loops and waits here until you manually enter a non-zero number to start the timer.

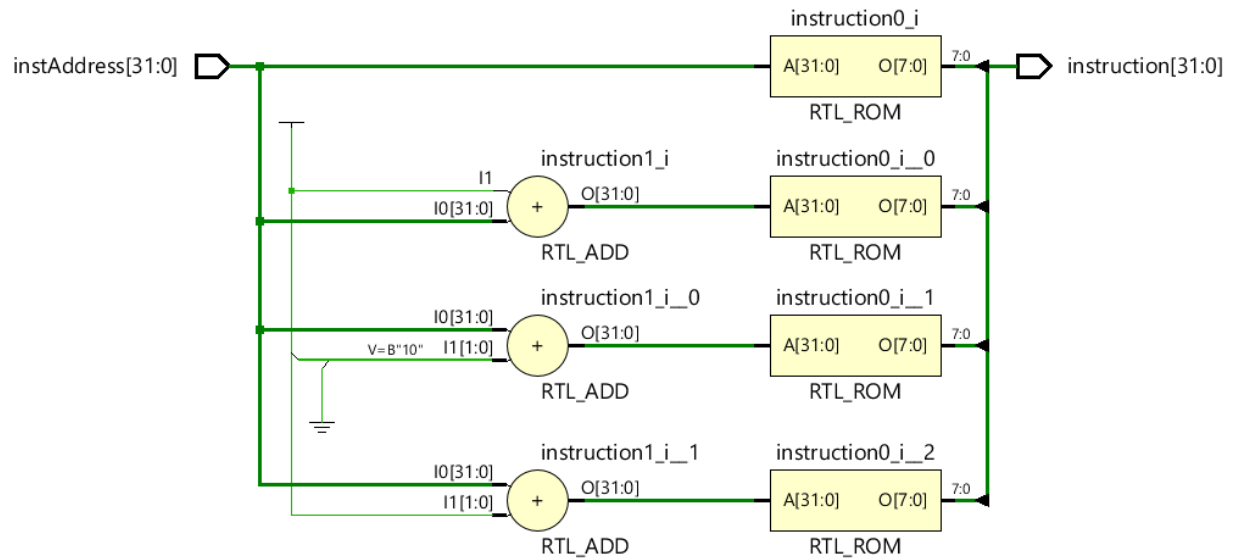
0x00000200 (Output): This is the "Display" address; you will see this value change from your input number down to zero as the countdown progresses.

0x000001F8 (Stack): This is temporary storage; it saves the return address and the current counter value so the CPU can resume the main loop after the timer finishes.

Decrementing Snaps

0x00000200	4	0	0	0
0x00000200	2	0	0	0
0x00000200	3	0	0	0
0x00000200	1	0	0	0
0x00000200	0	0	0	0

Task 3: Instruction Memory Module Design



The RTL schematic confirms the **byte-addressable** nature of the instruction memory. It shows three incrementer stages (RTL_ADD) used to calculate the addresses for bytes 1, 2, and 3 based on the base instAddress. These four bytes are fetched in parallel from the memory banks and concatenated to reconstruct the 32-bit machine code instruction.

```

1 module instructionMemory#(
2     parameter OPERAND_LENGTH = 31
3 ) (
4     input  [OPERAND_LENGTH:0] instAddress,
5     output reg [31:0] instruction
6 );
7
8 // Memory array: 256 bytes (64 total RISC-V instructions)
9 reg [7:0] memory [0:255];
10
11 // Combinational logic to fetch 4 bytes and concatenate them into a 32-bit word
12 // Logic follows Little-Endian: memory[n] is LSB, memory[n+3] is MSB
13 always @(*) begin
14     instruction = {memory[instAddress+3], memory[instAddress+2], memory[instAddress+1], memory[instAddress]};
15 end
16
17 initial begin
18     // --- Initialization ---
19     // addi x28, x0, 5 (Set constant 5)
20     memory[3]=8'h00; memory[2]=8'h50; memory[1]=8'h0E; memory[0]=8'h13;
21     // addi x2, x0, 511 (Initialize Stack Pointer)
22     memory[7]=8'h1F; memory[6]=8'hF0; memory[5]=8'h01; memory[4]=8'h13;
23     // addi x5, x0, 768 (MMIO Switch Base Address 0x300)
24     memory[11]=8'h30; memory[10]=8'h00; memory[9]=8'h02; memory[8]=8'h93;
25     // addi x6, x0, 512 (MMIO LED Base Address 0x200)
26     memory[15]=8'h20; memory[14]=8'h00; memory[13]=8'h03; memory[12]=8'h13;
27     // sw x28, 0(x5) (Store initial 5 into Switch address for simulation)
28     memory[19]=8'h01; memory[18]=8'hC2; memory[17]=8'hA0; memory[16]=8'h23;
29
30     // --- INPUT_WAIT ---
31     // sw x0, 0(x6) (Clear LEDs at start)
32     memory[23]=8'h00; memory[22]=8'h03; memory[21]=8'h20; memory[20]=8'h23;

```

```

33
34 // --- POLL_SWITCH ---
35 // lw x11, 0(x5) (Read Switches into x11)
36 memory[27]=8'h00; memory[26]=8'h02; memory[25]=8'hA5; memory[24]=8'h83;
37 // beq x11, x0, POLL_SWITCH (Loop if switch is 0)
38 memory[31]=8'hFE; memory[30]=8'h05; memory[29]=8'h8E; memory[28]=8'hE3;
39
40 // --- Prepare for Subroutine ---
41 // add x10, x11, x0 (Move input to x10 for argument passing)
42 memory[35]=8'h00; memory[34]=8'h05; memory[33]=8'h85; memory[32]=8'h33;
43 // jal x1, COUNTDOWN (Jump to Subroutine, save link in x1)
44 memory[39]=8'h00; memory[38]=8'h80; memory[37]=8'h00; memory[36]=8'hEF;
45 // beq x0, x0, INPUT_WAIT (Infinite loop back to start after return)
46 memory[43]=8'hFE; memory[42]=8'h00; memory[41]=8'h06; memory[40]=8'hE3;
47
48 // --- COUNTDOWN SUBROUTINE ---
49 // addi x2, x2, -8 (Adjust Stack Pointer)
50 memory[47]=8'hFF; memory[46]=8'h81; memory[45]=8'h01; memory[44]=8'h13;
51 // sw x1, 4(x2) (Save Return Address to Stack)
52 memory[51]=8'h00; memory[50]=8'h11; memory[49]=8'h22; memory[48]=8'h23;
53 // sw x12, 0(x2) (Save x12 to Stack)
54 memory[55]=8'h00; memory[54]=8'hC1; memory[53]=8'h20; memory[52]=8'h23;
55 // add x12, x10, x0 (Load initial count from x10)
56 memory[59]=8'h00; memory[58]=8'h05; memory[57]=8'h06; memory[56]=8'h33;
57
58 // --- COUNT_LOOP ---
59 // sw x12, 0(x6) (Write current count to LEDs)
60 memory[63]=8'h00; memory[62]=8'hC3; memory[61]=8'h20; memory[60]=8'h23;
61 // beq x12, x0, COUNT_DONE (Exit loop if count is zero)
62 memory[67]=8'h00; memory[66]=8'h06; memory[65]=8'h0C; memory[64]=8'h63;
63 // addi x12, x12, -1 (Decrement counter)
64 memory[71]=8'hFF; memory[70]=8'hF6; memory[69]=8'h06; memory[68]=8'h13;

```

```

65
66      // --- NESTED DELAY LOOP ---
67      // addi x13, x0, 3 (Set delay cycles)
68      memory[75]=8'h00; memory[74]=8'h30; memory[73]=8'h06; memory[72]=8'h93;
69      // addi x13, x13, -1 (Decrement delay)
70      memory[79]=8'hFF; memory[78]=8'hF6; memory[77]=8'h86; memory[76]=8'h93;
71      // bne x13, x0, DELAY (Loop back to delay start)
72      memory[83]=8'hFE; memory[82]=8'h06; memory[81]=8'h9E; memory[80]=8'hE3;
73
74      // beq x0, x0, COUNT_LOOP (Jump back to start of countdown)
75      memory[87]=8'hFE; memory[86]=8'h00; memory[85]=8'h04; memory[84]=8'hE3;
76
77      // --- COUNT_DONE (Cleanup) ---
78      // sw x0, 0(x6) (Ensure LEDs are 0)
79      memory[91]=8'h00; memory[90]=8'h03; memory[89]=8'h20; memory[88]=8'h23;
80      // lw x12, 0(x2) (Restore x12 from Stack)
81      memory[95]=8'h00; memory[94]=8'h01; memory[93]=8'h26; memory[92]=8'h03;
82      // lw x1, 4(x2) (Restore Return Address from Stack)
83      memory[99]=8'h00; memory[98]=8'h41; memory[97]=8'h20; memory[96]=8'h83;
84      // addi x2, x2, 8 (Restore Stack Pointer)
85      memory[103]=8'h00; memory[102]=8'h81; memory[101]=8'h01; memory[100]=8'h13;
86      // jalr x0, 0(x1) (Return to Main)
87      memory[107]=8'h00; memory[106]=8'h00; memory[105]=8'h80; memory[104]=8'h67;
88
89      // --- Program Exit/Idle ---
90      // j end (Infinite loop)
91      memory[111]=8'h00; memory[110]=8'h00; memory[109]=8'h00; memory[108]=8'h6F;
92      end
93 endmodule

```

Comments:

The **Instruction Memory module** was implemented as a 256-byte, byte-addressable ROM in Verilog to serve as the primary storage for the RISC-V assembly program. To bridge the gap between software and hardware, the assembly logic from Task 2 was manually translated into 32-bit hexadecimal machine code and loaded into the initial block using **Little-Endian** byte-ordering. The module utilizes combinational logic to fetch four consecutive 8-bit memory locations (indexed by the Program Counter (instAddress)) and concatenates them into a single 32-bit instruction for the CPU. This design successfully implements the FSM control flow, including the **Input-Waiting** and **Countdown** states, while facilitating memory-mapped I/O operations for switches at address 0x300 and LEDs at address 0x200.

TestBench

```
1 | `timescale 1ns / 1ps
2 |
3 | module tb_instructionMemory();
4 |     reg [31:0] instAddress;
5 |     wire [31:0] instruction;
6 |
7 |     // Instantiate the unit under test (UUT)
8 |     instructionMemory uut (
9 |         .instAddress(instAddress),
10 |        .instruction(instruction)
11 |    );
12 |
13 |    initial begin
14 |        // Monitor the output in the console
15 |        $monitor("Time: %0t | Address: %h | Instruction: %h", $time, instAddress, instruction);
16 |
17 |        // Test the first 5 instructions
18 |        instAddress = 32'h0; #10; // Should be 00500e13
19 |        instAddress = 32'h4; #10; // Should be 1ff00113
20 |        instAddress = 32'h8; #10; // Should be 30000293
21 |        instAddress = 32'hc; #10; // Should be 20000313
22 |        instAddress = 32'h10; #10; // Should be 01c2a023
23 |
24 |        $finish;
25 |    end
26 | endmodule
```

Verification(Console Output):

```
# run 1000ns
Time: 0 | Address: 00000000 | Instruction: 00500e13
Time: 10000 | Address: 00000004 | Instruction: 1ff00113
Time: 20000 | Address: 00000008 | Instruction: 30000293
Time: 30000 | Address: 0000000c | Instruction: 20000313
Time: 40000 | Address: 00000010 | Instruction: 01c2a023
$finish called at time : 50 ns : File "F:/lab10/lab10.srscs/sim_1/new/sim1.v" Line 24
xsim: Time (s): cpu = 00:00:13 ; elapsed = 00:00:06 . Memory (MB): peak = 1528.062 ; gain = 20.777
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_instructionMemory_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:18 ; elapsed = 00:00:33 . Memory (MB): peak = 1528.062 ; gain = 29.609
|
```

The verification of the **Instruction Memory Module** was conducted through a functional simulation to ensure the RISC-V processor could correctly fetch the FSM assembly instructions. By instantiating the module within a Verilog testbench, we simulated the Program Counter (PC) by cycling through memory addresses in **4-byte increments**.

The simulation results, as shown in the console output, confirm that the internal always block correctly performs **Little-Endian reconstruction**, concatenating four 8-bit registers into a single 32-bit instruction word. Specifically, at **Address 00000000**, the module successfully returned 00500e13 (the machine code for `addi x28, x0, 5`), and at **Address 00000010**, it returned 01c2a023 (the store instruction for the switch input). The alignment and data integrity across the tested range match the Task 2 assembly source exactly, proving that the instruction memory is properly initialized and ready to drive the state-based control flow of the FSM.

Name: Syed Faraz, Rayyan Amir Alvi	Student ID: sf0162, ra10104	Section: T2
---	------------------------------------	--------------------